CRRSS Integration Guide

Version 1.0.0 | **Phase 1B Stage 4: Build System Integration & Documentation**

Complete guide for integrating CRRSS into existing C projects with examples, best practices, and common patterns.

Table of Contents

Overview

CRRSS can be integrated into projects at multiple levels:

- **Build System Level**: Makefile/CMake integration for automated checks
- **Code Level**: Direct API usage for runtime analysis
- **Workflow Level**: Pre-commit hooks and CI/CD pipelines
- **IDE Level**: Editor integration for real-time feedback

Quick Integration

5-Minute Setup

```
# 1. Add CRRSS to your project
git submodule add <crrss-repo-url> tools/crrss

# 2. Build CRRSS
make -C tools/crrss all

# 3. Add to your Makefile
include tools/crrss/Makefile

# 4. Run analysis
make crrss-check
```

Build System Integration

Makefile Integration

Option 1: Include CRRSS Makefile

Add to your project's Makefile:

```
# ============================================================================
# CRRSS Integration
# ============================================================================

# CRRSS directories
CRRSS_DIR := tools/crrss
CRRSS_TOOL := $(CRRSS_DIR)/build/bin/crrss

# Include CRRSS targets
include $(CRRSS_DIR)/Makefile

# Add CRRSS to all target
all: $(TARGET) crrss

# Add CRRSS to clean target
clean: crrss-clean
    # your clean commands...

# Add CRRSS check to test target
test: $(TARGET) crrss-test
    # your test commands...
```

Option 2: Custom CRRSS Targets

```
# CRRSS Configuration
CRRSS_DIR := tools/crrss
CRRSS_TOOL := $(CRRSS_DIR)/build/bin/crrss
PROJECT_SOURCES := src/*.c include/*.h

# Build CRRSS
.PHONY: build-crrss
build-crrss:
    @$(MAKE) -C $(CRRSS_DIR) all

# Analyze project
.PHONY: analyze
analyze: build-crrss
    @echo "Analyzing project with CRRSS..."
    @$(CRRSS_TOOL) msm -d src/ --report analysis.html --format html
    @$(CRRSS_TOOL) stats --directory src/ --format json > stats.json

# Check code quality
.PHONY: quality
quality: build-crrss
    @$(CRRSS_TOOL) query --priority P0 --details
    @$(CRRSS_TOOL) query --priority P1 --details

# Pre-commit check
.PHONY: pre-commit
pre-commit: build-crrss
    @bash .git-hooks/pre-commit-crrss
```

Option 3: Makefile with Custom Rules

```
# CRRSS Integration with custom rules
CRRSS_DIR := tools/crrss
CRRSS_LIB := $(CRRSS_DIR)/build/lib/libcrrss.a

# Build CRRSS library
$(CRRSS_LIB):
    @$(MAKE) -C $(CRRSS_DIR) lib

# Link with CRRSS
$(TARGET): $(OBJS) $(CRRSS_LIB)
    $(CC) $(LDFLAGS) -o $@ $^ -L$(CRRSS_DIR)/build/lib -lcrrss -lm -lpthread

# Analyze on build
all: $(TARGET)
    @$(MAKE) analyze-quick

analyze-quick:
    @$(CRRSS_DIR)/build/bin/crrss msm -d src/ --max-issues 10
```

CMake Integration

Option 1: Add Subdirectory

```
# CMakeLists.txt

# Add CRRSS
add_subdirectory(tools/crrss)

# Link with CRRSS
target_link_libraries(${PROJECT_NAME} crrss)

# Custom targets
add_custom_target(analyze
    COMMAND ${CMAKE_CURRENT_SOURCE_DIR}/tools/crrss/build/bin/crrss msm -d ${CMAKE_SOU
RCE_DIR}/src
    COMMENT "Analyzing code with CRRSS"
    DEPENDS crrss_tool
)
```

Option 2: External Project

```
include(ExternalProject)

ExternalProject_Add(crrss
    SOURCE_DIR ${CMAKE_CURRENT_SOURCE_DIR}/tools/crrss
    CMAKE_ARGS -DCMAKE_INSTALL_PREFIX=${CMAKE_INSTALL_PREFIX}
    BUILD_COMMAND make
    INSTALL_COMMAND ""
)

# Use CRRSS library
add_dependencies(${PROJECT_NAME} crrss)
target_link_libraries(${PROJECT_NAME}
    ${CMAKE_CURRENT_SOURCE_DIR}/tools/crrss/build/lib/libcrrss.a
    m pthread
)
```

Option 3: Find Package

```
# After installing CRRSS to system
find_package(CRRSS REQUIRED)

target_link_libraries(${PROJECT_NAME} CRRSS::crrss)
```

Autotools Integration

```
# configure.ac

# Check for CRRSS
AC_CHECK_LIB([crrss], [msm_initialize],
    [CRRSS_LIBS="-lcrrss"],
    [AC_MSG_ERROR([CRRSS library not found])]
)

AC_SUBST([CRRSS_LIBS])

# Add to Makefile.am
AM_CPPFLAGS = -I$(top_srcdir)/tools/crrss
LDADD = $(top_builddir)/tools/crrss/build/lib/libcrrss.a $(CRRSS_LIBS)
```

Code Integration

Basic Integration

1. Initialize CRRSS

```
#include "msm/msm.h"

// Global MSM context
msm_context_t* g_msm_ctx = NULL;

int initialize_analysis(void) {
    msm_config_t config = {
        .enable_runtime_tracking = true,
        .enable_static_analysis = true,
        .enable_stack_traces = true,
        .max_tracked_allocations = 10000
    };

    g_msm_ctx = msm_initialize(&config);
    if (!g_msm_ctx) {
        fprintf(stderr, "Failed to initialize MSM\n");
        return -1;
    }

    return 0;
}

void cleanup_analysis(void) {
    if (g_msm_ctx) {
        msm_shutdown(g_msm_ctx);
        g_msm_ctx = NULL;
    }
}
```

2. Wrap Memory Allocations

```c
#include "msm/msm.h"

void* checked_malloc(size_t size, const char* file, int line) {
    void* ptr = malloc(size);
    if (ptr && g_msm_ctx) {
        msm_track_allocation(g_msm_ctx, ptr, size, file, line);
    }
    return ptr;
}

void checked_free(void* ptr, const char* file, int line) {
    if (ptr && g_msm_ctx) {
        msm_track_deallocation(g_msm_ctx, ptr, file, line);
    }
    free(ptr);
}

// Use macros for convenience
#define MALLOC(size) checked_malloc(size, __FILE__, __LINE__)
#define FREE(ptr) checked_free(ptr, __FILE__, __LINE__)
```

3. Analyze at Runtime

```c
void perform_analysis(void) {
    if (!g_msm_ctx) return;

    msm_analysis_result_t result = {0};

    // Analyze current state
    if (msm_runtime_analysis(g_msm_ctx, &result) == CRRSS_SUCCESS) {
        printf("Runtime Analysis Results:\n");
        printf("  Total issues: %u\n", result.total_issues);
        printf("  Memory leaks: %u\n", result.memory_leaks);
        printf("  Use-after-free: %u\n", result.use_after_free);

        // Generate report
        msm_report_config_t report_config = {
            .format = MSM_REPORT_FORMAT_TEXT,
            .include_stack_traces = true
        };

        msm_generate_report(g_msm_ctx, &result,
                            "runtime_analysis.txt", &report_config);
    }
}
```

Advanced Integration

Memory Manager Integration

```
// memory_manager.h
#ifndef MEMORY_MANAGER_H
#define MEMORY_MANAGER_H

#include "msm/msm.h"

typedef struct {
    msm_context_t* msm;
    size_t total_allocated;
    size_t total_freed;
    uint32_t allocation_count;
} memory_manager_t;

memory_manager_t* memory_manager_create(void);
void memory_manager_destroy(memory_manager_t* mm);
void* memory_manager_alloc(memory_manager_t* mm, size_t size,
                           const char* file, int line);
void memory_manager_free(memory_manager_t* mm, void* ptr,
                         const char* file, int line);
void memory_manager_report(memory_manager_t* mm);

#endif // MEMORY_MANAGER_H
```

```c
// memory_manager.c
#include "memory_manager.h"
#include <stdlib.h>
#include <string.h>

memory_manager_t* memory_manager_create(void) {
    memory_manager_t* mm = malloc(sizeof(memory_manager_t));
    if (!mm) return NULL;

    memset(mm, 0, sizeof(memory_manager_t));

    msm_config_t config = {
        .enable_runtime_tracking = true,
        .enable_static_analysis = false,  // Runtime only
        .enable_stack_traces = true,
        .max_tracked_allocations = 10000
    };

    mm->msm = msm_initialize(&config);
    if (!mm->msm) {
        free(mm);
        return NULL;
    }

    return mm;
}

void memory_manager_destroy(memory_manager_t* mm) {
    if (!mm) return;

    // Generate final report
    memory_manager_report(mm);

    if (mm->msm) {
        msm_shutdown(mm->msm);
    }

    free(mm);
}

void* memory_manager_alloc(memory_manager_t* mm, size_t size,
                           const char* file, int line) {
    if (!mm) return NULL;

    void* ptr = malloc(size);
    if (ptr) {
        mm->total_allocated += size;
        mm->allocation_count++;

        if (mm->msm) {
            msm_track_allocation(mm->msm, ptr, size, file, line);
        }
    }

    return ptr;
}

void memory_manager_free(memory_manager_t* mm, void* ptr,
                         const char* file, int line) {
    if (!mm || !ptr) return;

    if (mm->msm) {
```

```
        msm_track_deallocation(mm->msm, ptr, file, line);
    }

    free(ptr);
    mm->total_freed += 1;  // Size not tracked for simplicity
}

void memory_manager_report(memory_manager_t* mm) {
    if (!mm) return;

    printf("=== Memory Manager Report ===\n");
    printf("Total allocated: %zu bytes\n", mm->total_allocated);
    printf("Allocation count: %u\n", mm->allocation_count);

    if (mm->msm) {
        msm_analysis_result_t result = {0};
        if (msm_runtime_analysis(mm->msm, &result) == CRRSS_SUCCESS) {
            printf("Memory leaks: %u\n", result.memory_leaks);
            printf("Use-after-free: %u\n", result.use_after_free);
            printf("Double-free: %u\n", result.double_free);
        }
    }
}
```

Usage Example

```
#include "memory_manager.h"

#define MM_ALLOC(mm, size) memory_manager_alloc(mm, size, __FILE__, __LINE__)
#define MM_FREE(mm, ptr) memory_manager_free(mm, ptr, __FILE__, __LINE__)

int main(void) {
    // Create memory manager
    memory_manager_t* mm = memory_manager_create();
    if (!mm) {
        fprintf(stderr, "Failed to create memory manager\n");
        return 1;
    }

    // Use memory manager
    char* buffer = MM_ALLOC(mm, 1024);
    if (buffer) {
        strcpy(buffer, "Hello, CRRSS!");
        printf("%s\n", buffer);
        MM_FREE(mm, buffer);
    }

    // Cleanup (generates report automatically)
    memory_manager_destroy(mm);

    return 0;
}
```

Development Workflow

Recommended Workflow

```
1. Development
   ↓
2. Local Analysis (make crrss-check)
   ↓
3. Fix Issues
   ↓
4. Pre-commit Hook (automated)
   ↓
5. Commit
   ↓
6. CI/CD Analysis
   ↓
7. Code Review
   ↓
8. Merge
```

Daily Development

```bash
# Morning: Update and check
git pull
make crrss

# During development: Quick checks
make crrss-analyze FILE=src/new_feature.c

# Before commit: Full analysis
make crrss-check
git add .
git commit -m "Add feature"  # Pre-commit hook runs

# End of day: Generate report
make crrss-check > daily_report.txt
```

Feature Development

```
# Start feature
git checkout -b feature/new-memory-manager

# Develop with continuous checking
while developing:
    # Write code
    vim src/memory_manager.c

    # Quick check
    make crrss-analyze FILE=src/memory_manager.c

    # Fix issues
    # Repeat

# Final check before PR
make crrss-check
make crrss-test

# Create PR
git push origin feature/new-memory-manager
```

CI/CD Integration

GitHub Actions

```yaml
# .github/workflows/crrss.yml
name: CRRSS Analysis

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main, develop ]

jobs:
  crrss-analysis:
    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v3
      with:
        submodules: true

    - name: Install dependencies
      run: |
        sudo apt-get update
        sudo apt-get install -y gcc make

    - name: Build CRRSS
      run: make crrss

    - name: Run CRRSS Tests
      run: make crrss-test

    - name: Analyze Codebase
      run: |
        make crrss-check > crrss_report.txt || true
        cat crrss_report.txt

    - name: Upload Report
      uses: actions/upload-artifact@v3
      with:
        name: crrss-report
        path: crrss_report.txt

    - name: Check for Critical Issues
      run: |
        if grep -q "Critical issues: [^0]" crrss_report.txt; then
          echo "Critical issues found!"
          exit 1
        fi
```

GitLab CI

```yaml
# .gitlab-ci.yml
stages:
  - build
  - test
  - analyze

build_crrss:
  stage: build
  script:
    - make crrss
  artifacts:
    paths:
      - tools/crrss/build/

test_crrss:
  stage: test
  dependencies:
    - build_crrss
  script:
    - make crrss-test

analyze_codebase:
  stage: analyze
  dependencies:
    - build_crrss
  script:
    - make crrss-check > crrss_report.txt || true
    - cat crrss_report.txt
  artifacts:
    paths:
      - crrss_report.txt
    reports:
      codequality: crrss_report.json
  allow_failure: true
```

Jenkins

```
// Jenkinsfile
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                checkout scm
                sh 'git submodule update --init --recursive'
            }
        }

        stage('Build CRRSS') {
            steps {
                sh 'make crrss'
            }
        }

        stage('Test CRRSS') {
            steps {
                sh 'make crrss-test'
            }
        }

        stage('Analyze Code') {
            steps {
                sh 'make crrss-check > crrss_report.txt || true'
                archiveArtifacts artifacts: 'crrss_report.txt', fingerprint: true
            }
        }

        stage('Quality Gate') {
            steps {
                script {
                    def report = readFile('crrss_report.txt')
                    if (report.contains('Critical issues: [^0]')) {
                        error('Critical issues found in CRRSS analysis')
                    }
                }
            }
        }
    }

    post {
        always {
            publishHTML([
                reportDir: '.',
                reportFiles: 'crrss_report.txt',
                reportName: 'CRRSS Analysis Report'
            ])
        }
    }
}
```

IDE Integration

VS Code

Create `.vscode/tasks.json`:

```
{
    "version": "2.0.0",
    "tasks": [
        {
            "label": "CRRSS: Analyze Current File",
            "type": "shell",
            "command": "make",
            "args": [
                "crrss-analyze",
                "FILE=${file}"
            ],
            "problemMatcher": [],
            "group": {
                "kind": "build",
                "isDefault": false
            }
        },
        {
            "label": "CRRSS: Check Project",
            "type": "shell",
            "command": "make",
            "args": ["crrss-check"],
            "problemMatcher": [],
            "group": {
                "kind": "build",
                "isDefault": false
            }
        },
        {
            "label": "CRRSS: Generate Report",
            "type": "shell",
            "command": "bash",
            "args": [
                "-c",
                "make crrss-check && code crrss_report.txt"
            ],
            "problemMatcher": []
        }
    ]
}
```

Vim/Neovim

Add to `.vimrc` or `init.vim`:

```
" CRRSS integration
command! CRRSSAnalyze :!make crrss-analyze FILE=%
command! CRRSSCheck :!make crrss-check
command! CRRSSReport :!make crrss-check > crrss_report.txt && vim crrss_report.txt

" Key bindings
nnoremap <leader>ca :CRRSSAnalyze<CR>
nnoremap <leader>cc :CRRSSCheck<CR>
nnoremap <leader>cr :CRRSSReport<CR>
```

CLion/IntelliJ

Add External Tool:

1. File → Settings → Tools → External Tools
2. Add new tool:
   - Name: CRRSS Analyze
   - Program: make
   - Arguments: crrss-analyze FILE=$FilePath$
   - Working directory: $ProjectFileDir$

Best Practices

1. Start Early

Integrate CRRSS from the beginning of your project:

```
# New project setup
mkdir my_project && cd my_project
git init
git submodule add <crrss-url> tools/crrss
make crrss
./.git-hooks/install-hooks.sh install
```

2. Continuous Checking

Run CRRSS frequently during development:

```
# After each file change
make crrss-analyze FILE=src/changed_file.c

# Before each commit
make crrss-check
```

3. Incremental Analysis

Analyze only changed files for faster feedback:

```
# Get changed files
CHANGED_FILES=$(git diff --name-only HEAD | grep '\.c$')

# Analyze each
for file in $CHANGED_FILES; do
    make crrss-analyze FILE=$file
done
```

4. Prioritize Issues

Focus on critical issues first:

```
# Check P0 issues
crrss query --priority P0

# Then P1
crrss query --priority P1
```

5. Track Progress

Generate periodic reports:

```
# Weekly report
make crrss-check > reports/week_$(date +%Y%m%d).txt

# Compare with previous
diff reports/week_20251004.txt reports/week_20251011.txt
```

6. Team Standards

Establish team-wide CRRSS configuration:

```
# .crrss (project root)
[global]
verbose = false

[sciv]
strict_mode = true
max_complexity = 15

[msm]
enable = true
runtime_tracking = true
```

7. Documentation

Document CRRSS usage in your project:

```
# README.md

## Code Quality

We use CRRSS for code analysis:

```bash
# Analyze code
make crrss-check

# Before committing
make crrss-analyze FILE=your_file.c
```

Common Patterns

Pattern 1: Memory Safety Wrapper

```c
// safe_memory.h
#ifndef SAFE_MEMORY_H
#define SAFE_MEMORY_H

#include <stddef.h>

void* safe_malloc(size_t size, const char* file, int line);
void* safe_calloc(size_t nmemb, size_t size, const char* file, int line);
void* safe_realloc(void* ptr, size_t size, const char* file, int line);
void safe_free(void* ptr, const char* file, int line);

#define SAFE_MALLOC(size) safe_malloc(size, __FILE__, __LINE__)
#define SAFE_CALLOC(nmemb, size) safe_calloc(nmemb, size, __FILE__, __LINE__)
#define SAFE_REALLOC(ptr, size) safe_realloc(ptr, size, __FILE__, __LINE__)
#define SAFE_FREE(ptr) safe_free(ptr, __FILE__, __LINE__)

#endif
```
```

Pattern 2: Automatic Analysis

```
// Auto-analyze on program exit
#include <atexit.h>
#include "msm/msm.h"

void auto_analyze(void) {
    if (g_msm_ctx) {
        msm_analysis_result_t result = {0};
        msm_runtime_analysis(g_msm_ctx, &result);

        if (result.total_issues > 0) {
            msm_report_config_t config = {
                .format = MSM_REPORT_FORMAT_TEXT
            };
            msm_generate_report(g_msm_ctx, &result,
                                "exit_analysis.txt", &config);
            printf("Analysis report saved to exit_analysis.txt\n");
        }
    }
}

int main(void) {
    initialize_analysis();
    atexit(auto_analyze);

    // Your code...

    return 0;
}
```

Pattern 3: Conditional Analysis

```
// Enable analysis only in debug builds
#ifdef DEBUG
    #define CRRSS_ENABLED 1
#else
    #define CRRSS_ENABLED 0
#endif

#if CRRSS_ENABLED
    #define TRACK_ALLOC(ptr, size) \
        msm_track_allocation(g_msm_ctx, ptr, size, __FILE__, __LINE__)
    #define TRACK_FREE(ptr) \
        msm_track_deallocation(g_msm_ctx, ptr, __FILE__, __LINE__)
#else
    #define TRACK_ALLOC(ptr, size) ((void)0)
    #define TRACK_FREE(ptr) ((void)0)
#endif
```

Performance Considerations

1. Analysis Frequency

```
# Fast: Analyze only changed files
make crrss-analyze FILE=src/changed.c

# Medium: Analyze specific directory
crrss msm -d src/subsystem/

# Slow: Full project analysis
make crrss-check
```

2. Caching

Enable caching for faster repeated analysis:

```
export CRRSS_CACHE_SIZE=5000
export CRRSS_MAX_THREADS=8
```

3. Incremental Mode

Analyze incrementally in large projects:

```
# Day 1: Analyze module A
crrss msm -d src/module_a/ --report module_a.txt

# Day 2: Analyze module B
crrss msm -d src/module_b/ --report module_b.txt

# Day 3: Combine reports
cat module_*.txt > full_report.txt
```

Migration Guide

Existing Project Integration

Step 1: Add CRRSS

```
cd /path/to/your/project
git submodule add <crrss-url> tools/crrss
```

Step 2: Build CRRSS

```
make -C tools/crrss all
```

Step 3: Add Makefile Targets

Add to your Makefile:

```
# CRRSS Integration
CRRSS_DIR := tools/crrss
CRRSS_TOOL := $(CRRSS_DIR)/build/bin/crrss

.PHONY: analyze
analyze:
    @$(MAKE) -C $(CRRSS_DIR) all
    @$(CRRSS_TOOL) msm -d src/ --report analysis.txt
```

Step 4: Baseline Analysis

```
# Create baseline
make analyze > baseline_analysis.txt

# Review issues
less baseline_analysis.txt
```

Step 5: Fix Critical Issues

```
# Focus on P0/P1 issues first
grep -E "Priority: P[01]" baseline_analysis.txt > critical_issues.txt
```

Step 6: Integrate into Workflow

```
# Install pre-commit hook
./.git-hooks/install-hooks.sh install

# Add to CI/CD
# (See CI/CD Integration section)
```

Examples

Complete Integration Example

```
# project/
# ├── src/
# ├── include/
# ├── tests/
# ├── tools/
# │   └── crrss/
# ├── Makefile
# └── .git-hooks/
```

```
# Makefile
PROJECT := myproject
SOURCES := $(wildcard src/*.c)
HEADERS := $(wildcard include/*.h)
CRRSS_DIR := tools/crrss
CRRSS_TOOL := $(CRRSS_DIR)/build/bin/crrss

all: $(PROJECT) analyze-quick

$(PROJECT): $(SOURCES)
    $(CC) $(CFLAGS) -o $@ $^

# CRRSS Targets
.PHONY: build-crrss analyze analyze-quick quality ci-check

build-crrss:
    @$(MAKE) -C $(CRRSS_DIR) all

analyze: build-crrss
    @$(CRRSS_TOOL) msm -d src/ --report full_analysis.html --format html
    @$(CRRSS_TOOL) stats --directory src/ --format json > stats.json

analyze-quick: build-crrss
    @$(CRRSS_TOOL) msm -d src/ --max-issues 10

quality: build-crrss
    @$(CRRSS_TOOL) query --priority P0 --details
    @$(CRRSS_TOOL) query --priority P1 --details | head -20

ci-check: build-crrss
    @$(CRRSS_TOOL) msm -d src/ --report ci_report.txt --format text
    @if grep -q "Critical issues: [^0]" ci_report.txt; then exit 1; fi

test: $(PROJECT)
    ./$(PROJECT) --test

clean:
    rm -f $(PROJECT) *.o
    @$(MAKE) -C $(CRRSS_DIR) clean
```

Support

For integration issues:
- Check `tools/crrss/docs/USAGE.md`
- Run `make crrss-help`
- See examples in `tools/crrss/examples/`
- Open issue in repository

CRRSS Integration Guide - Version 1.0.0
Phase 1B Stage 4: Build System Integration & Documentation
Last Updated: October 11, 2025